Customer Segmentation using K-Means Clustering

Objective

The objective of this project is to segment customers into distinct groups based on their behavior such as tenure and spending patterns. This helps in understanding user behavior and enables targeted business strategies.

Importing libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Loading dataset

```
df = pd.read_csv("/content/Telco-Customer-Churn.csv")
df.head()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneServi
0	7590-VHVEG	Female	0	Yes	No	1	N
1	5575-GNVDE	Male	0	No	No	34	Y
2	3668-QPYBK	Male	0	No	No	2	Y
3	7795-CFOCW	Male	0	No	No	45	N
4	9237-HQITU	Female	0	No	No	2	Y

5 rows × 21 columns

Data Overview

We explore the dataset structure, data types, and summary statistics.

```
df.shape
df.info()
```

```
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

	SeniorCitizen	tenure	MonthlyCharges
count	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692
std	0.368612	24.559481	30.090047
min	0.000000	0.000000	18.250000
25%	0.000000	9.000000	35.500000
50%	0.000000	29.000000	70.350000
75%	0.000000	55.000000	89.850000
max	1.000000	72.000000	118.750000

Data Cleaning

- Removed unnecessary columns
- Converted data types
- Handled missing values

```
df.drop("customerID", axis=1, inplace=True)
```

```
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
df["TotalCharges"] = df["TotalCharges"].fillna(df["TotalCharges"].median())
```

Encoding Categorical Variables

Categorical variables are converted into numerical format using one-hot encoding.

```
df = pd.get_dummies(df, drop_first=True)
```

Feature Selection

Selected features:

- Tenure
- Monthly Charges
- Total Charges

```
features = ["tenure", "MonthlyCharges", "TotalCharges"]
X = df[features]
```

Feature Scaling

Data is standardized using StandardScaler to ensure equal importance of features.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Finding Optimal Clusters (Elbow Method)

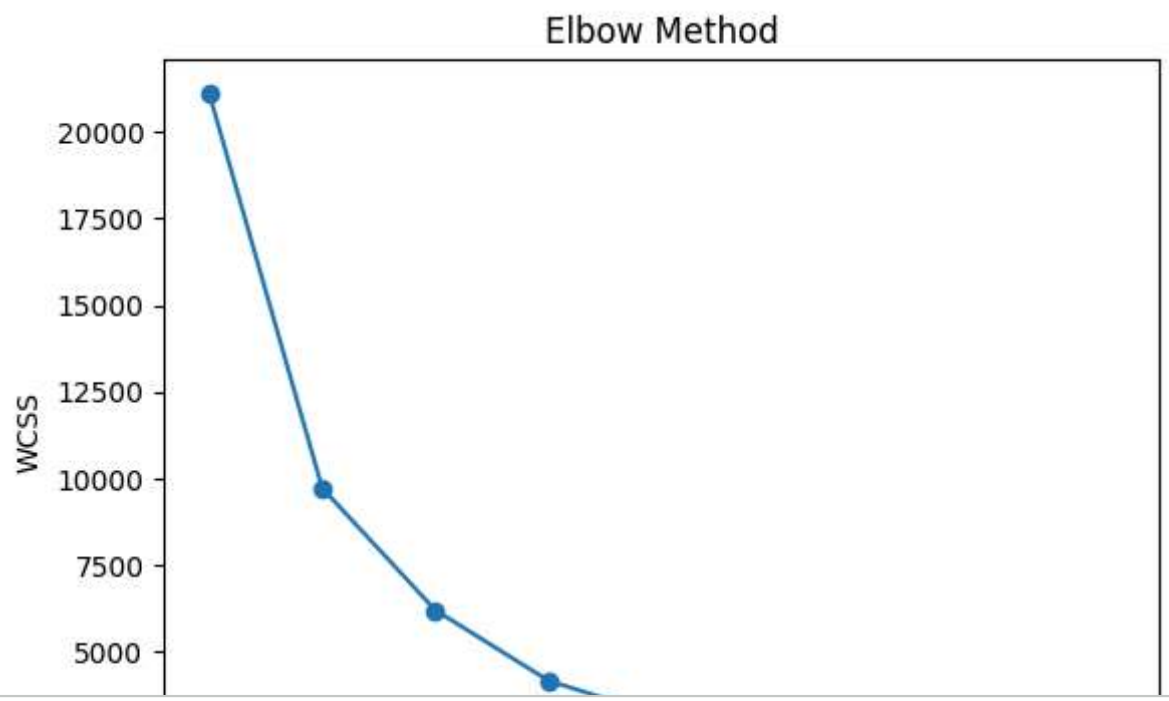
The Elbow Method helps determine the optimal number of clusters by analyzing WCSS.

```
from sklearn.cluster import KMeans

wcss = []

for i in range(1, 10):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.plot(range(1, 10), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Clusters")
plt.ylabel("WCSS")
plt.show()
```



Applying K-Means Clustering

K-Means clustering is applied to group customers into segments.

```
kmeans = KMeans(n_clusters=3, random_state=42)
df["Cluster"] = kmeans.fit_predict(X_scaled)
```

Cluster Visualization (PCA)

PCA is used to reduce dimensions and visualize clusters in 2D space.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(6,5))
plt.scatter(X_pca[:,0], X_pca[:,1], c=df["Cluster"])
plt.title("Customer Segments")
plt.xlabel("PCA1")
plt.ylabel("PCA2")
plt.show()
```

Cluster Interpretation

- Cluster 0: Loyal Customers Customers with high tenure and high spending. These users are highly engaged and contribute significantly to revenue.
- Engagement Customers Customers with low tenure and low spending. These users are either new or not actively using the service.
- Cluster 2: At-Risk Customers Customers with high spending but low tenure. These users are likely to churn if not retained.

Cluster Profiling

Analyzing average feature values for each cluster.

```
df.groupby("Cluster")[features].mean()
```

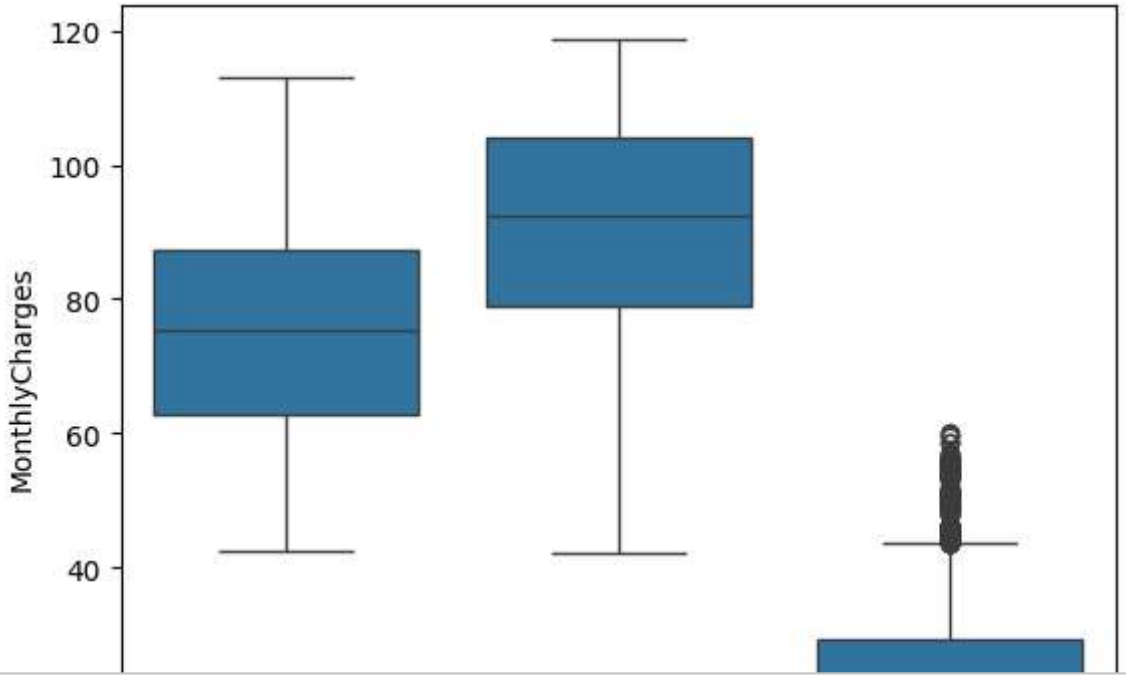
Cluster	tenure	MonthlyCharges	TotalCharges
0	13.267164	75.073619	1036.212771
1	58.572273	89.672432	5245.974864
2	29.392048	26.648151	810.608414

```
df.to_csv("segmented_customers.csv", index=False)
```

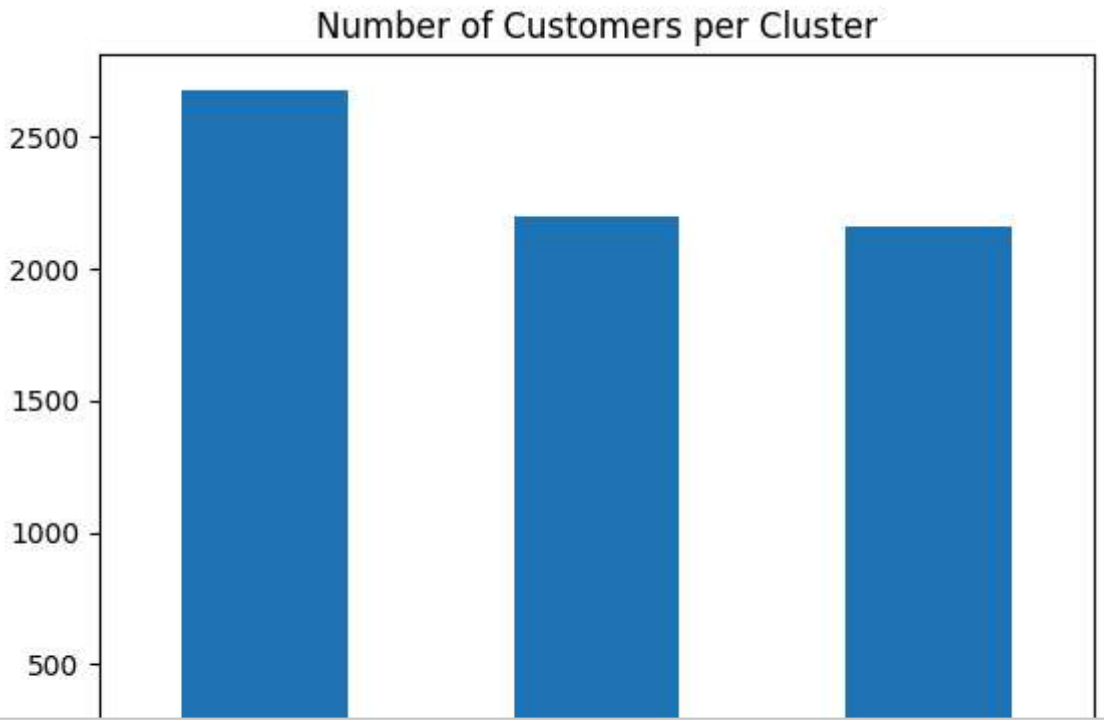
Visual Analysis

- Distribution of customers across clusters
- Spending patterns across clusters
- Relationships between features

```
sns.boxplot(x="Cluster", y="MonthlyCharges", data=df)
plt.show()
```

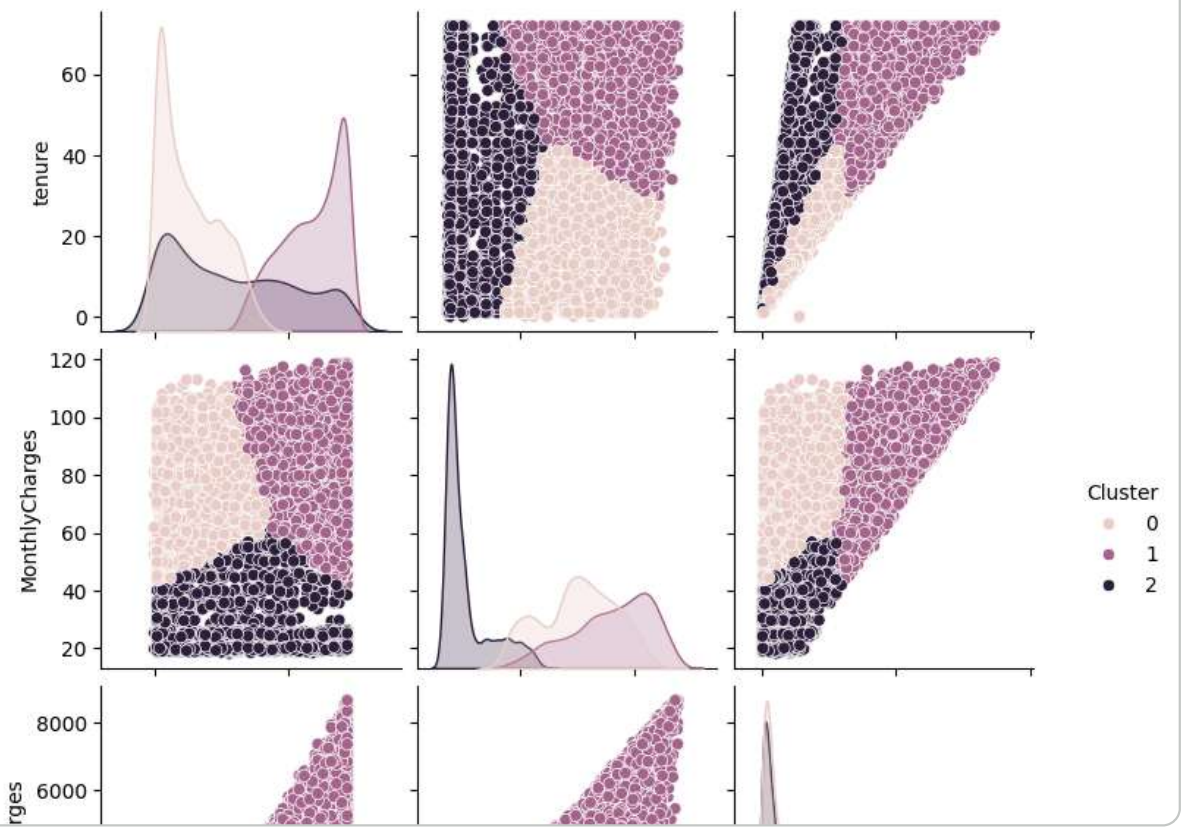



```
df["Cluster"].value_counts().plot(kind='bar')
plt.title("Number of Customers per Cluster")
plt.show()
```




```
import seaborn as sns
```

```
sns.pairplot(df, vars=["tenure", "MonthlyCharges", "TotalCharges"], hue="Clu
plt.show()
```



Churn Analysis by Cluster

Analyzing churn rate across clusters to identify high-risk segments.

```
df.groupby("Cluster")["Churn_Yes"].mean()
```

Cluster	Churn_Yes
0	0.470896
1	0.153636
2	0.124364

dtype: float64

```
(df.groupby("Cluster")["Churn_Yes"].mean()*100)
```

Cluster	Churn_Yes
0	47.089552
1	15.363636
2	12.436431

dtype: float64

Key Business Insights

- Cluster 0 represents loyal and high-value customers
- Cluster 1 includes low engagement users
- Cluster 2 contains high-paying users with high churn risk

Business Recommendations

- Retain loyal customers through rewards and loyalty programs
- Improve engagement for low-activity users
- Target at-risk customers with personalized offers and retention strategies

```
cluster_profile = df.groupby("Cluster")[["tenure","MonthlyCharges","TotalCha
cluster_profile
```

Cluster	tenure	MonthlyCharges	TotalCharges
0	13.267164	75.073619	1036.212771
1	58.572273	89.672432	5245.974864
2	29.392048	26.648151	810.608414

Next steps: Generate code with `cluster_profile` New interactive sheet

```
cluster_profile["count"] = df["Cluster"].value_counts()
cluster_profile
```

Cluster	tenure	MonthlyCharges	TotalCharges	count
0	13.267164	75.073619	1036.212771	2680
1	58.572273	89.672432	5245.974864	2200
2	29.392048	26.648151	810.608414	2163

Next steps: Generate code with cluster_profile New interactive sheet

```
cluster_profile = df.groupby("Cluster")[["tenure","Mon
cluster_profile["count"] = df["Cluster"].value_counts(
cluster_profile
```

Cluster	tenure	MonthlyCharges	TotalCharges	count
0	13.267164	75.073619	1036.212771	2680
1	58.572273	89.672432	5245.974864	2200
2	29.392048	26.648151	810.608414	2163

Next steps: Generate code with cluster_profile New interactive sheet

Conclusion

Customer segmentation enables businesses to understand user behavior and take